Manejo y visualización de datos

Data Wrangling

Data wrangling

Los datos del mundo real tienen errores, inconsistencias o están estructurados para la entrada de datos — no para el análisis.

Data wrangling: transformar datos “en bruto” en un formato adecuado para la visualización, el modelado y el análisis.

Toma información que puede estar implícita y la transforma en tablas que den esa información de manera explícita.

Fuente:

Data wrangling

Incluye la ***limpieza / preprocesamiento / transformación*** de datos

Fuente:

Datos tabulares

Matriz rectangular que organiza valores en filas y columnas.

flight	time_hour	origin	origin_name	dest	dest_name	tailnum	plane_model	carrier	carrier_name	temp	wind
1545	2026-03-01 09:00	JFK	John F. Kennedy Intl	LAX	Los Angeles Intl	N542UA	Boeing 737	UA	United Airlines	42.1	12.4
1714	2026-03-01 09:00	JFK	John F. Kennedy Intl	SFO	San Francisco Intl	N806AS	Airbus A320	AS	Alaska Airlines	42.1	12.4
2109	2026-03-01 09:00	JFK	John F. Kennedy Intl	MIA	Miami Intl	N542UA	Boeing 737	UA	United Airlines	42.1	12.4
1142	2026-03-01 09:00	JFK	John F. Kennedy Intl	ORD	Chicago O'Hare Intl	N14228	Boeing 737	UA	United Airlines	42.1	12.4
461	2026-03-01 10:00	LGA	LaGuardia Airport	ORD	Chicago O'Hare Intl	N14228	Boeing 737	UA	United Airlines	39.5	15.2
583	2026-03-01 10:00	EWR	Newark Liberty Intl	LAX	Los Angeles Intl	N542UA	Boeing 737	UA	United Airlines	40.0	10.1

Fuente: https://r4ds.hadley.nz

Datos tabulares

Almacenar "lo similar con lo similar" (ej., todas las fechas en una columna, todos los precios en otra) es **eficiente**

- **Vectorización**: tener variables en columnas permite a lenguajes de programación como R hacer **operaciones vectorizadas**: en vectores enteros de valores simultáneamente
  (ej., dividir una columna de distancia por una columna de tiempo para calcular la velocidad para todas las filas a la vez)

Fuente:

Data frames y tibbles

El **data frame** es la estructura principal de R para almacenar datos tabulares

A nivel interno, un data frame es una **lista de vectores de la misma longitud**

Cada lista puede diferentes tipos de datos (texto, números, fechas) en diferentes columnas, a diferencia de una matriz (*matrix*)

Fuente:

Data frames y tibbles

Un ***tibble*** es un tipo especial de data frame de los paquetes del **tidyverse**

- Los data frames estándar pueden imprimir miles de filas y saturar la consola — éstos muestran las primeras filas y las columnas
- Muestran explícitamente el tipo de datos de cada columna (ej., `<int>`, `<chr>`, `<dbl>`) directamente bajo el nombre de la variable
- Tienen más advertencias sobre posibles problemas que los data frames tradicionales.

Fuente:

Los *verbos* del data wrangling

Para manipular datos tabulares (*wrangle*), usamos un conjunto específico de "**verbos**" — funciones que describen la acción:

- *select()*
- *filter()*
- *mutate()*
- *arrange()*
- *summarize()*
- etc.

Se pueden encadenar usando el **operador pipe** (|> ó %>%), volviendo al código más legible.

Fuente:

Operaciones sobre filas

Filter

Mantiene las filas que coinciden con una condición lógica y descarta el resto.

flights_full |> filter(origin == "JFK" | temp < 32)

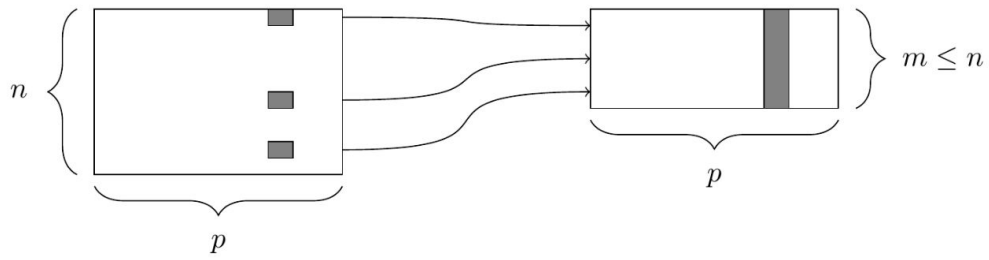


Figure 4.1: The `filter()` function. At left, a data frame that contains matching entries in a certain column for only a subset of the rows. At right, the resulting data frame after filtering.

Fuente: https://mdsr-book.github.io/

Operaciones sobre filas

Arrange

Cambia el orden de las filas basándose en valores de las columnas — no elimina datos

flights_full |> arrange(desc(wind))

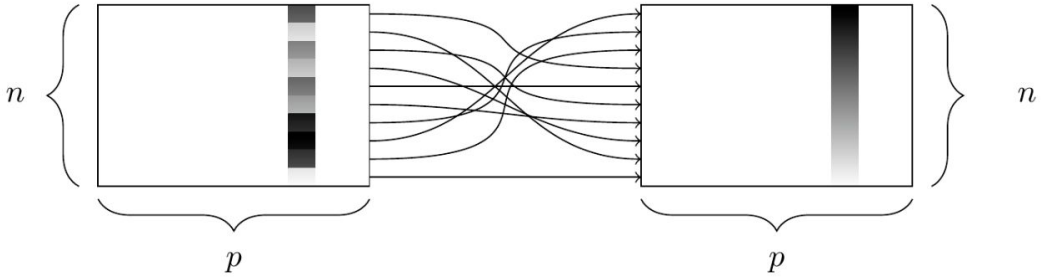


Figure 4.4: The `arrange()` function. At left, a data frame with an ordinal variable. At right, the resulting data frame after sorting the rows in descending order of that variable.

Fuente: https://mdsr-book.github.io/

Operaciones en columnas

Select

Mantiene solo columnas específicas.

flights_full |> select(flight, carrier_name, origin, dest)

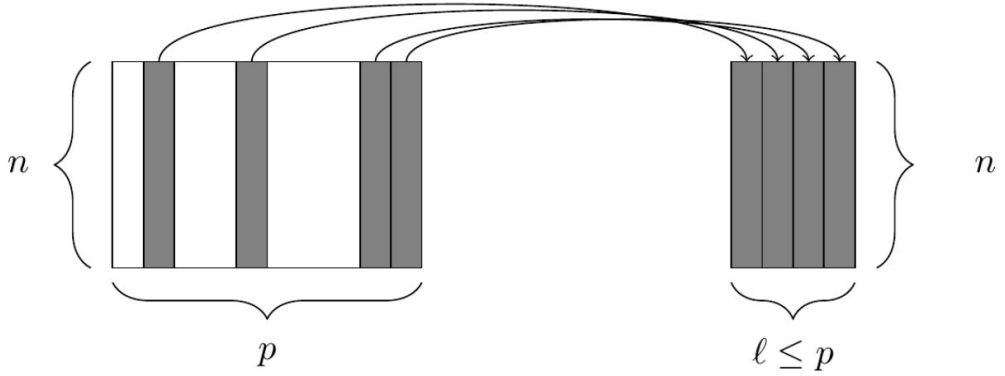


Figure 4.2: The `select()` function. At left, a data frame, from which we retrieve only a few of the columns. At right, the resulting data frame after selecting those columns.

Fuente: https://mdsr-book.github.io/

Operaciones en columnas

Mutate

Crea nuevas columnas al final del dataset que pueden ser funciones de columnas existentes.

*flights_full |> mutate(temp_celsius = (temp - 32) * 5/9)*

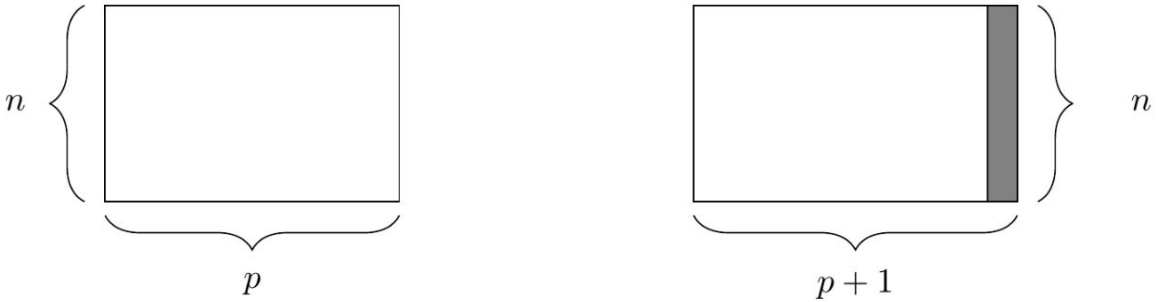
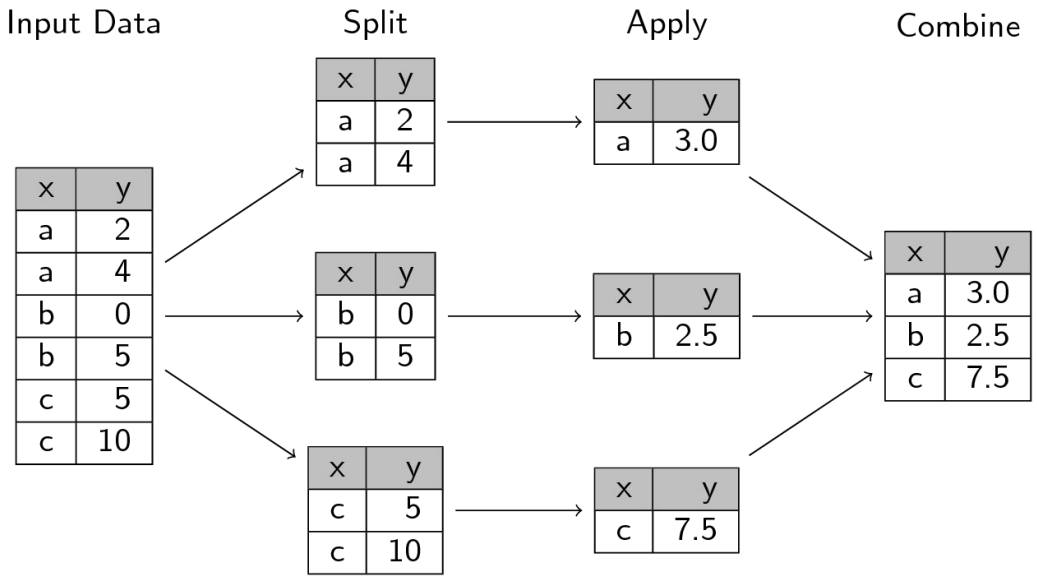


Figure 4.3: The `mutate()` function. At right, the resulting data frame after adding a new column.

Fuente: https://mdsr-book.github.io/

Agregaciones

Split-Apply-Combine



Fuente: https://r-cubed-intermediate.rostools.org/sessions/split-apply-combine

Agregaciones

Split-Apply-Combine

1. Group By: divide el dataset en grupos basándose en una variable
2. Summarize: aplica una función a cada grupo para reducirlo a una sola fila.

```
flights_full |>
    group_by(carrier_name) |>
    summarize(
        avg_wind = mean(wind, na.rm = T)
    )
```

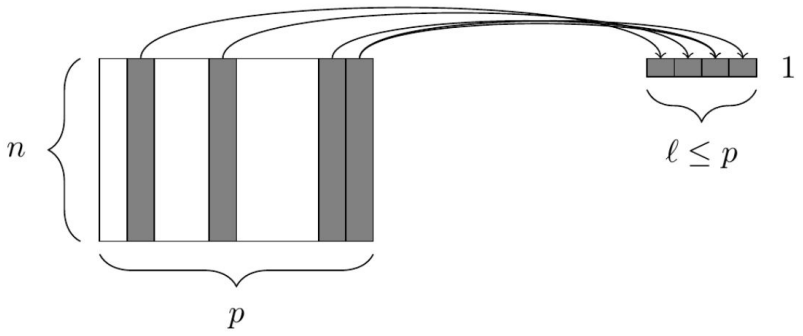
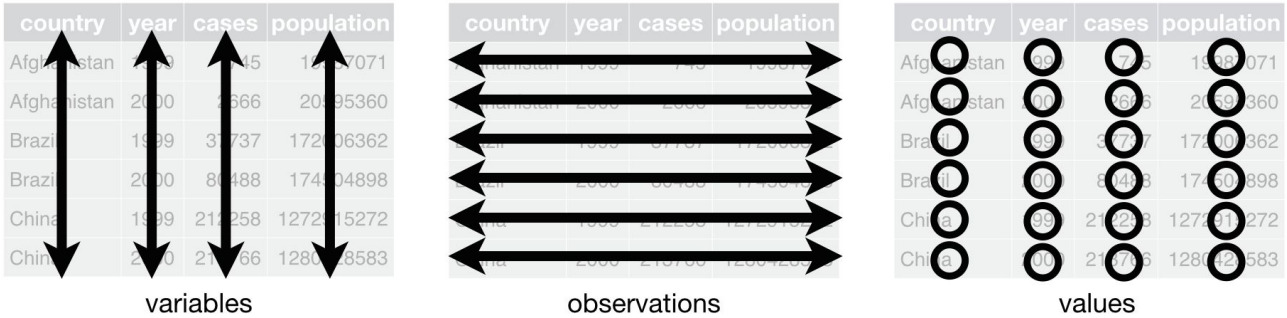


Figure 4.5: The `summarize()` function. At left, a data frame. At right, the resulting data frame after aggregating four of the columns.

Fuente: https://mdsr-book.github.io/

Tidy data

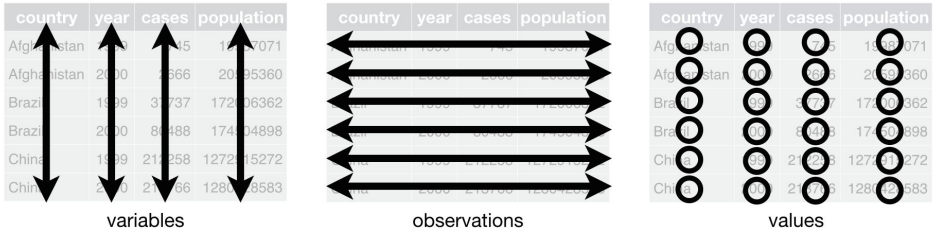
1. Las **variables** son **columnas**: cada columna representa un atributo
2. Las **observaciones** (casos) son **filas**: cada fila representa una instancia única y específica (ej., una persona, un vuelo o un país-año).
3. Los **valores** son **celdas**: cada celda contiene una sola medición



Fuente: https://r4ds.hadley.nz/

Tidy data

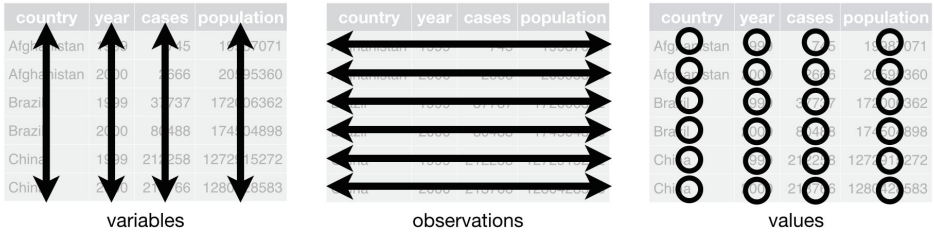
- Tener variables en columnas permite que las **funciones vectorizadas** de R funcionen de manera natural



Fuente: https://r4ds.hadley.nz/

Tidy data

- Tener variables en columnas permite que las **funciones vectorizadas** de R funcionen de manera natural
- **Escalabilidad**: es más fácil agregar una nueva variable como columna
  - Si tengo una tabla de *filas: países*, *columas: años*, *celdas: PBI*
  - Si quiero agregar una segunda medición (e.g. población), necesito una nueva tabla o cambiar el formato
  - En cambio si la tabla original tiene las columnas: país, año, PBI: agregar un atributo es sencillamente agregar una columna sin cambiar el formato



Fuente: https://r4ds.hadley.nz/

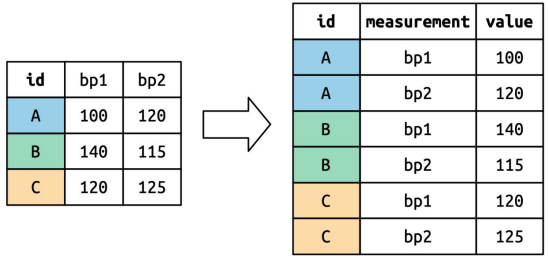
Formato *long*

Hacer que los datos sean "largos"

Toma nombres de columnas y los convierte en una sola columna.

→ Aumenta el número de filas y disminuye el número de columnas.

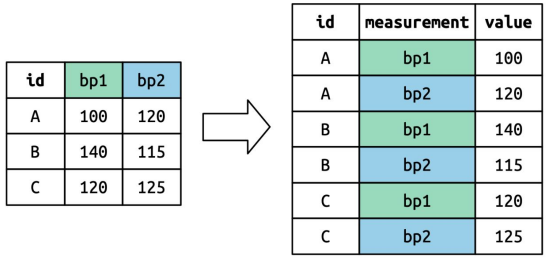
→ Sirve para hacer visualizaciones con *ggplot2*



id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

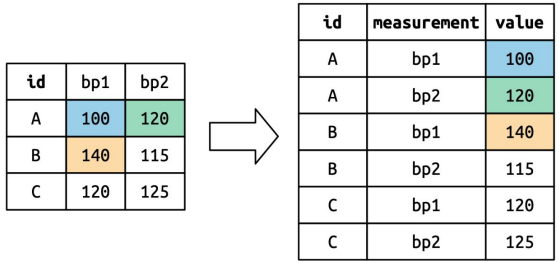
Figure 5.3: Columns that are already variables need to be repeated, once for each column that is pivoted.



id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Figure 5.4: The column names of pivoted columns become values in a new column. The values need to be repeated once for each row of the original dataset.



id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Figure 5.5: The number of values is preserved (not repeated), but unwound row-by-row.

Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
who2
#> # A tibble: 7,240 × 58
#>   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp_m_4554
#>   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
#> 1 Afghanistan  1980       NA        NA        NA        NA        NA
#> 2 Afghanistan  1981       NA        NA        NA        NA        NA
#> 3 Afghanistan  1982       NA        NA        NA        NA        NA
#> 4 Afghanistan  1983       NA        NA        NA        NA        NA
#> 5 Afghanistan  1984       NA        NA        NA        NA        NA
#> 6 Afghanistan  1985       NA        NA        NA        NA        NA
#> # ℹ 7,234 more rows
#> # ℹ 51 more variables: sp_m_5564 <dbl>, sp_m_65 <dbl>, sp_f_014 <dbl>, …
```

sp/rel/ep: method used for tuberculosis diagnosis
m/f: binary gender
014/1524/2534/3544/4554/5564/65: age range

Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
who2
#> # A tibble: 7,240 × 58
#>   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp_m_4554
#>   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
#> 1 Afghanistan  1980       NA        NA        NA        NA        NA
#> 2 Afghanistan  1981       NA        NA        NA        NA        NA
#> 3 Afghanistan  1982       NA        NA        NA        NA        NA
#> 4 Afghanistan  1983       NA        NA        NA        NA        NA
#> 5 Afghanistan  1984       NA        NA        NA        NA        NA
#> 6 Afghanistan  1985       NA        NA        NA        NA        NA
#> # ℹ 7,234 more rows
#> # ℹ 51 more variables: sp_m_5564 <dbl>, sp_m_65 <dbl>, sp_f_014 <dbl>, …
```

```
who2 |>
  pivot_longer(
    cols = !(country:year),
    names_to = c("diagnosis", "gender", "age"),
    names_sep = "_",
    values_to = "count"
  )
#> # A tibble: 405,440 × 6
#>   country      year diagnosis gender age   count
#>   <chr>       <dbl> <chr>     <chr>  <chr> <dbl>
#> 1 Afghanistan  1980 sp        m      014      NA
#> 2 Afghanistan  1980 sp        m      1524     NA
#> 3 Afghanistan  1980 sp        m      2534     NA
#> 4 Afghanistan  1980 sp        m      3544     NA
#> 5 Afghanistan  1980 sp        m      4554     NA
#> 6 Afghanistan  1980 sp        m      5564     NA
#> # ℹ 405,434 more rows
```


Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
who2
#> # A tibble: 7,240 × 58
#>   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp_m_4554
#>   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
#> 1 Afghanistan  1980       NA        NA        NA        NA        NA
#> 2 Afghanistan  1981       NA        NA        NA        NA        NA
#> 3 Afghanistan  1982       NA        NA        NA        NA        NA
#> 4 Afghanistan  1983       NA        NA        NA        NA        NA
#> 5 Afghanistan  1984       NA        NA        NA        NA        NA
#> 6 Afghanistan  1985       NA        NA        NA        NA        NA
#> # ℹ 7,234 more rows
#> # ℹ 51 more variables: sp_m_5564 <dbl>, sp_m_65 <dbl>, sp_f_014 <dbl>, …
```

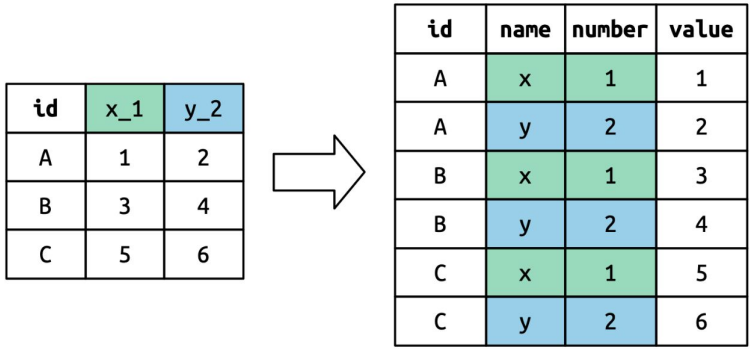


Figure 5.6: Pivoting columns with multiple pieces of information in the names means that each column name now fills in values in multiple output columns.

Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
household
#> # A tibble: 5 × 5
#>   family dob_child1 dob_child2 name_child1 name_child2
#>    <int> <date>     <date>     <chr>       <chr>
#> 1      1 1998-11-26 2000-01-29 Susan       Jose
#> 2      2 1996-06-22 NA         Mark        <NA>
#> 3      3 2002-07-11 2004-04-05 Sam         Seth
#> 4      4 2004-10-10 2009-08-27 Craig       Khai
#> 5      5 2000-12-05 2005-02-28 Parker      Gracie
```


Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
household
#> # A tibble: 5 × 5
#>   family dob_child1 dob_child2 name_child1 name_child2
#>    <int> <date>     <date>     <chr>       <chr>
#> 1      1 1998-11-26 2000-01-29 Susan       Jose
#> 2      2 1996-06-22 NA         Mark        <NA>
#> 3      3 2002-07-11 2004-04-05 Sam         Seth
#> 4      4 2004-10-10 2009-08-27 Craig       Khai
#> 5      5 2000-12-05 2005-02-28 Parker      Gracie
```

Luego de separar por "_", la segunda parte determina los valores de la nueva col. "child"

```
household |>
  pivot_longer(
    cols = !family,
    names_to = c(".value", "child"),
    names_sep = "_",
    values_drop_na = TRUE
  )
#> # A tibble: 9 × 4
#>   family child  dob        name
#>    <int> <chr>  <date>     <chr>
#> 1      1 child1 1998-11-26 Susan
#> 2      1 child2 2000-01-29 Jose
#> 3      2 child1 1996-06-22 Mark
#> 4      3 child1 2002-07-11 Sam
#> 5      3 child2 2004-04-05 Seth
#> 6      4 child1 2004-10-10 Craig
#> # ℹ 3 more rows
```


Fuente: https://r4ds.hadley.nz

Formato *long*

A veces los nombres de las columnas tienen valores

```
household
#> # A tibble: 5 × 5
#>   family dob_child1 dob_child2 name_child1 name_child2
#>    <int> <date>     <date>     <chr>       <chr>
#> 1      1 1998-11-26 2000-01-29 Susan       Jose
#> 2      2 1996-06-22 NA         Mark        <NA>
#> 3      3 2002-07-11 2004-04-05 Sam         Seth
#> 4      4 2004-10-10 2009-08-27 Craig       Khai
#> 5      5 2000-12-05 2005-02-28 Parker      Gracie
```

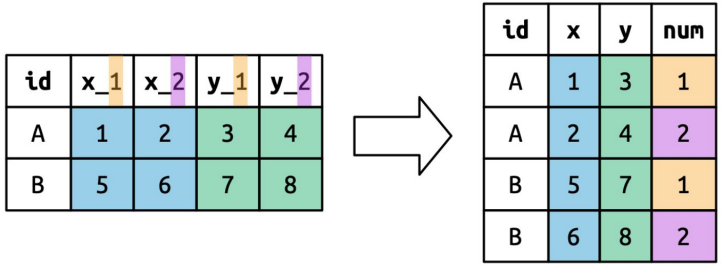


Figure 5.7: Pivoting with `names_to = c(".value", "num")` splits the column names into two components: the first part determines the output column name (`x` or `y`), and the second part determines the value of the `num` column.

Fuente: https://r4ds.hadley.nz

Formato *wide*

A veces, una observación está en muchas filas
e.g. un mismo paciente con una fila para "altura" y otra para "peso"

Podemos pasar de *long* a *wide*, tal que una fila equivalga a un caso completo (ej., una fila por paciente con cols. para “altura” y “peso”),

→ a veces es más fácil para leer tablas o calcular diferencias entre dos métricas.

Formato *wide*

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Fuente: https://www.thedataschool.co.uk/luke-bennett/long-vs-wide-data-tables/

Formato *wide*

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Fuente: https://www.thedataschool.co.uk/luke-bennett/long-vs-wide-data-tables/

long y *wide* son nociones *relativas*

longer

dni	año	mate	lengua	geo	física
111	2020	8	7	9	6
111	2021	9	8	8	7
222	2020	6	9	7	8
222	2021	7	10	8	9

wider

Fuente:

long y *wide* son nociones *relativas*

dni	año	mate	lengua	geo	física
111	2020	8	7	9	6
111	2021	9	8	8	7
222	2020	6	9	7	8
222	2021	7	10	8	9

longer

dni	año	materia	nota
111	2020	mate	8
111	2020	lengua	7
111	2020	geo	9
111	2020	física	6
111	2021	mate	9
111	2021	lengua	8
111	2021	geo	8
111	2021	física	7
222	2020	mate	6
222	2020	lengua	9
222	2020	geo	7
222	2020	física	8
222	2021	mate	7
222	2021	lengua	10
222	2021	geo	8
222	2021	física	9

wider

dni	mate_2020	mate_2021	lengua_2020	lengua_2021	geo_2020	geo_2021	física_202
111	8	9	7	8	9	8	6
222	6	7	9	10	7	8	8

Fuente:

¿Sirve tener toda la información en una tabla grande?
Por ejemplo, un dataset de vuelos:

flight	time_hour	origin	origin_name	dest	dest_name	tailnum	plane_model	carrier	carrier_name	temp	wind
1545	2026-03-01 09:00	JFK	John F. Kennedy Intl	LAX	Los Angeles Intl	N542UA	Boeing 737	UA	United Airlines	42.1	12.4
1714	2026-03-01 09:00	JFK	John F. Kennedy Intl	SFO	San Francisco Intl	N806AS	Airbus A320	AS	Alaska Airlines	42.1	12.4
2109	2026-03-01 09:00	JFK	John F. Kennedy Intl	MIA	Miami Intl	N542UA	Boeing 737	UA	United Airlines	42.1	12.4
1142	2026-03-01 09:00	JFK	John F. Kennedy Intl	ORD	Chicago O'Hare Intl	N14228	Boeing 737	UA	United Airlines	42.1	12.4
461	2026-03-01 10:00	LGA	LaGuardia Airport	ORD	Chicago O'Hare Intl	N14228	Boeing 737	UA	United Airlines	39.5	15.2
583	2026-03-01 10:00	EWR	Newark Liberty Intl	LAX	Los Angeles Intl	N542UA	Boeing 737	UA	United Airlines	40.0	10.1

Fuente:

Trabajando con muchas tablas

Es muy **ineficiente** almacenar datos repetitivos en *cada* fila de un registro (e.g. el nombre legal completo de una aerolínea)

e.g. conviene almacenar un código en los datos de vuelos y los detalles completos en una tabla separada de "airlines"

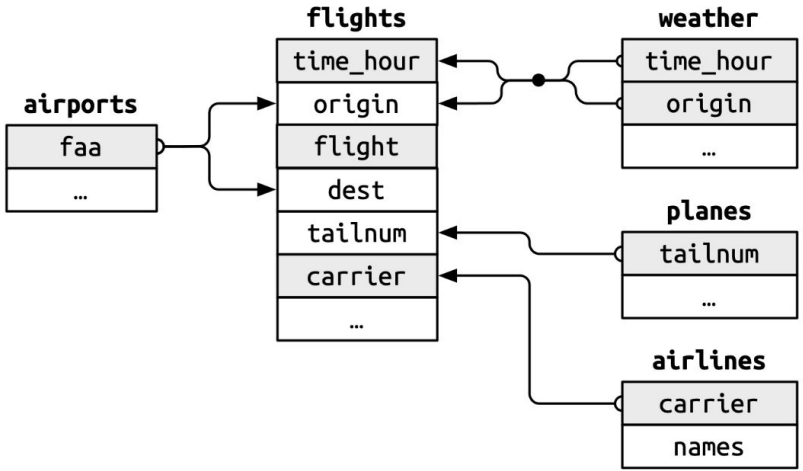


Figure 19.1: Connections between all five data frames in the nycflights13 package. Variables making up a primary key are colored grey, and are connected to their corresponding foreign keys with arrows.

Fuente: https://r4ds.hadley.nz

Trabajando con muchas tablas

→ Separar los datos en muchas tablas reduce el tamaño de los datos
→ si es necesario actualizar datos, es mucho más **veloz y seguro** (e.g. si una aerolínea cambia de nombre)

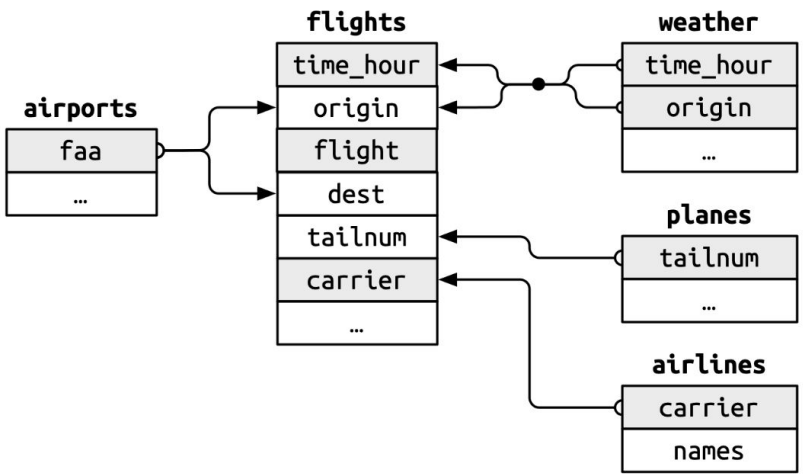


Figure 19.1: Connections between all five data frames in the nycflights13 package. Variables making up a primary key are colored grey, and are connected to their corresponding foreign keys with arrows.

Fuente: https://r4ds.hadley.nz

Trabajando con muchas tablas

→ Separar los datos en muchas tablas reduce el tamaño de los datos
→ si es necesario actualizar datos, es mucho más **veloz y seguro** (e.g. si una aerolínea cambia de nombre)

Usamos **Joins** para combinar estas tablas y responder preguntas específicas,
e.g. vincular el retraso de un vuelo a la velocidad del viento en el aeropuerto de origen

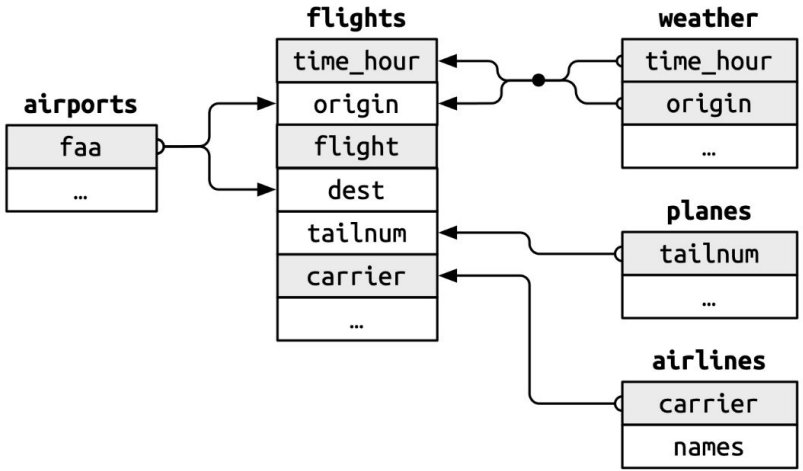


Figure 19.1: Connections between all five data frames in the nycflights13 package. Variables making up a primary key are colored grey, and are connected to their corresponding foreign keys with arrows.

Fuente: https://r4ds.hadley.nz

Primary keys (PKs) y foreign keys (FKs)

Para unir tablas, necesitamos identificadores comunes: **keys / claves**

PK: una variable o conjunto de variables que identifica de manera única una observación *en su propia tabla*

- e.g. `tailnum` es la PK en la tabla `planes`; identifica unívocamente un avión específico
- Cuando una sola variable no es suficiente, se llama “compound key” / clave compuesta

FK: una variable que corresponde a una clave primaria en *otra tabla*

- e.g. `tailnum` aparece en la tabla `flights` como una FK para vincular un vuelo a un avión específico.

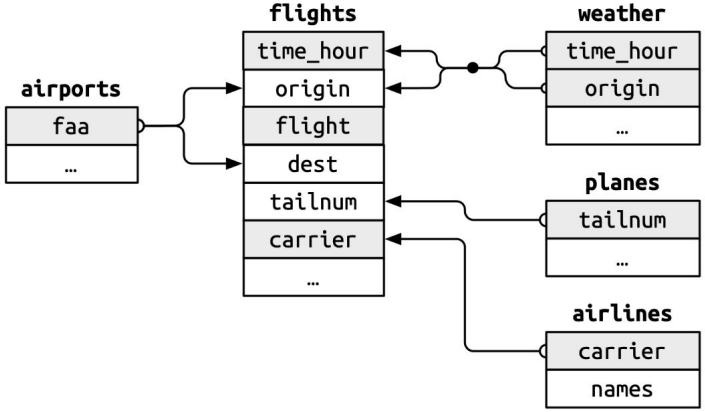


Figure 19.1: Connections between all five data frames in the nycflights13 package. Variables making up a primary key are colored grey, and are connected to their corresponding foreign keys with arrows.

Fuente: https://r4ds.hadley.nz

Joins

Los **mutating joins** emparejan observaciones por sus claves y copian variables de una tabla a otra.

Inner Join empareja pares de observaciones siempre que sus claves sean iguales.

→ mantiene solo las filas que existen en ambas tablas
→ las filas que no coinciden se eliminan del resultado.

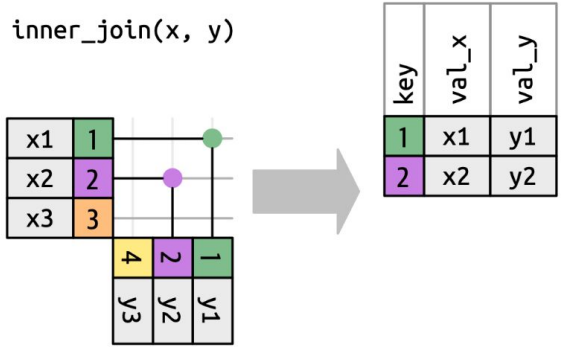


Figure 19.4: An inner join matches each row in `x` to the row in `y` that has the same value of `key`. Each match becomes a row in the output.

Fuente: https://r4ds.hadley.nz

Joins

Left Join mantiene todas las observaciones en la primera tabla. Si no hay coincidencia en la segunda tabla, llena las nuevas columnas con NA (valores faltantes).

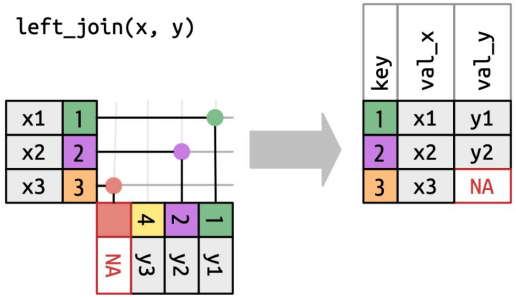


Figure 19.5: A visual representation of the left join where every row in `x` appears in the output.

Fuente: https://r4ds.hadley.nz

Joins

Left Join mantiene todas las observaciones en la primera tabla. Si no hay coincidencia en la segunda tabla, llena las nuevas columnas con NA (valores faltantes).

Right Join mantiene todas las observaciones en la segunda tabla

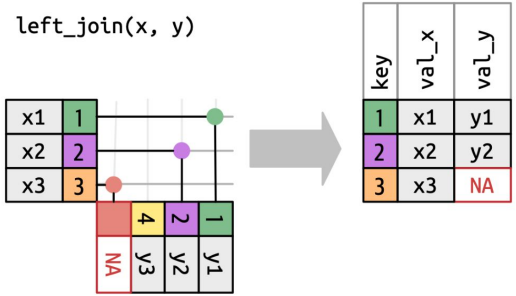


Figure 19.5: A visual representation of the left join where every row in `x` appears in the output.

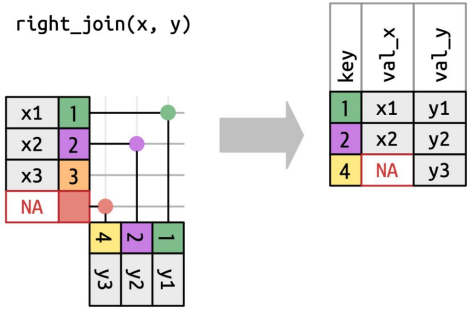


Figure 19.6: A visual representation of the right join where every row of `y` appears in the output.

Fuente: https://r4ds.hadley.nz

Joins

Left Join mantiene todas las observaciones en la primera tabla. Si no hay coincidencia en la segunda tabla, llena las nuevas columnas con NA (valores faltantes).

Right Join mantiene todas las observaciones en la segunda tabla

Full Join mantiene todas las observaciones de ambas tablas, llenando con `NA` donde no haya coincidencia.

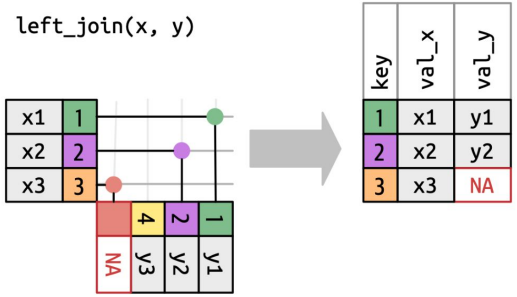


Figure 19.5: A visual representation of the left join where every row in `x` appears in the output.

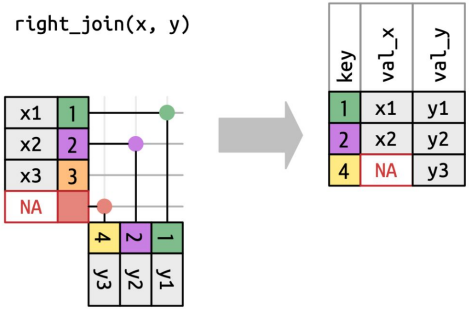


Figure 19.6: A visual representation of the right join where every row of `y` appears in the output.

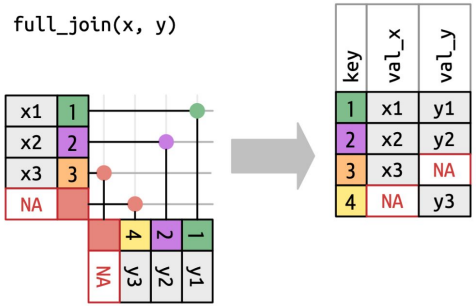
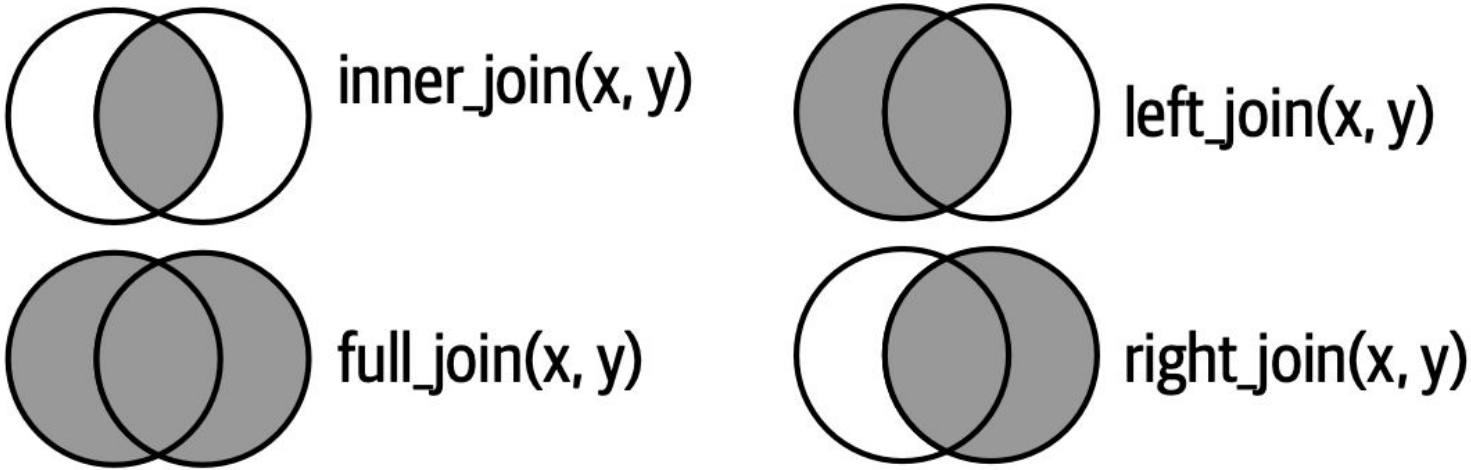


Figure 19.7: A visual representation of the full join where every row in `x` and `y` appears in the output.

Fuente: https://r4ds.hadley.nz

Joins



Fuente: https://r4ds.hadley.nz

Joins con IDs repetidos

left_join(a, b, by=x) — ¿qué pasa si b tiene duplicados en x?

Fuente: https://r4ds.hadley.nz

Joins con IDs repetidos

Ejemplo: left join

Uno-a-Muchos (One-to-Many): una fila en la tabla X coincide con muchas filas en la tabla Y. La fila de X se duplica para cada coincidencia en Y.

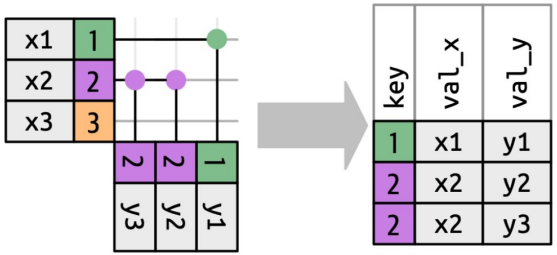


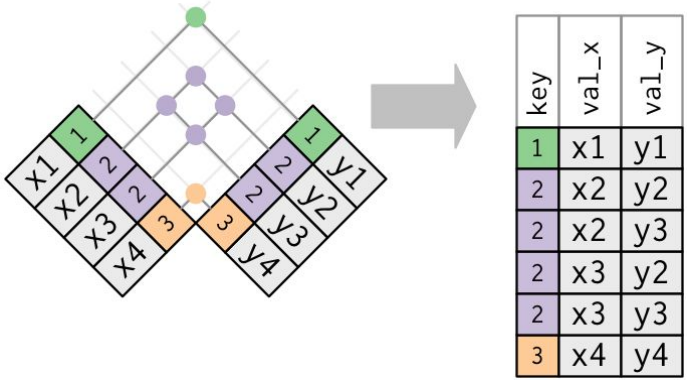
Figure 19.9: The three ways a row in `x` can match. `x1` matches one row in `y`, `x2` matches two rows in `y`, `x3` matches zero rows in y. Note that while there are three rows in `x` and three rows in the output, there isn't a direct correspondence between the rows.

Fuente: https://r4ds.hadley.nz

Joins con IDs repetidos

Ejemplo: left join

Muchos-a-Muchos (Many-to-Many): si ambas tablas tienen duplicados, crea todas las combinaciones posibles — a veces es un signo de un error (ej., intentar unir solo por `year` cuando se necesita `country` y `year`).



Fuente: https://r4ds.hadley.nz

Joins con IDs repetidos

Ejemplo: left join

Si no hay valores repetidos en las claves, el match es **Uno-a-Uno (One-to-One)**: Cada fila en la tabla X coincide exactamente con una fila en la tabla Y. La salida tiene el mismo número de filas que X.

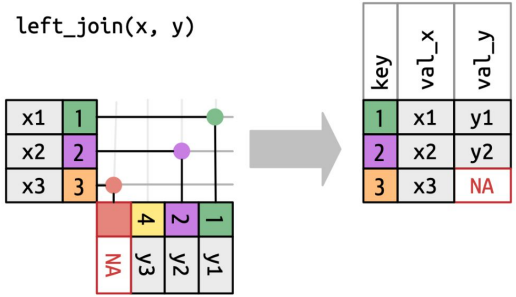


Figure 19.5: A visual representation of the left join where every row in `x` appears in the output.

Fuente: https://r4ds.hadley.nz

Joins con IDs repetidos

Verificar las dimensiones y las claves con frecuencia!

Por ejemplo,
- verificar que las claves primarias sean únicas antes de un join, si es lo que se espera
- verificar el nro. de filas después de los joins

Claves y estandarización

Fuentes de datos distintas pueden usar convenciones de nomenclatura distintas.

- "Bolivia (Estado Plurinacional de)" vs. "Bolivia"
- "Antigua & Barbuda" vs. "Antigua and Barbuda"

Fuente:

Claves y estandarización

Fuentes de datos distintas pueden usar convenciones de nomenclatura distintas.

- "Bolivia (Estado Plurinacional de)" vs. "Bolivia"
- "Antigua & Barbuda" vs. "Antigua and Barbuda"

Sin estandarización, se ven como entidades completamente diferentes, haciendo imposible el join.

Hay librerías que ayudan a la **estandarización de códigos**

- e.g. `countrycode` puede traducir varias convenciones de nomenclatura de países en un solo estándar.

Fuente:

Fuzzy matching

Podemos usar algoritmos que calculan la **distancia entre strings** para hacer **fuzzy matching**: emparejamiento *no exacto*

Fuente:

Fuzzy matching

Podemos usar algoritmos que calculan la **distancia entre strings** para hacer **fuzzy matching**: emparejamiento *no exacto*

- Distancia de strings:
  cuántas ediciones se requieren para convertir el texto A en el texto B.
- Emparejar strings con distancia baja (alta similitud)
- A veces requiere supervisión humana para confirmar los pares
  - "Bolivia (Estado Plurinacional de)" — "Bolivia" ✅
  - "Estados Unidos" — "Emiratos Árabes Unidos" ❌

Fuente:

Tipos de datos

- cómo se almacenan datos en la computadora?
- qué operaciones son válidas sobre los datos?

Fuente:

Valores lógicos (Booleanos)

`TRUE`, `FALSE`, o `NA`

- `TRUE` se trata como 1, y `FALSE` se almacena/trata como 0.
- El costo de almacenamiento es extremadamente bajo (en el límite 1 bit, pero R lo guarda como un entero)

Fuente:

Valores lógicos (Booleanos)

`TRUE`, `FALSE`, o `NA`

- `TRUE` se trata como 1, y `FALSE` se almacena/trata como 0.
- El costo de almacenamiento es extremadamente bajo (en el límite 1 bit, pero R lo guarda como un entero)

Operaciones

- Comparación: `==`, `>`, `<`, `!=`.
- Álgebra booleana: `&` (AND), `|` (OR), y `!` (NOT).
- Resúmenes:
  e.g. `sum(x)` cuenta el número de `TRUE`
  e.g. `mean(x)` calcula la proporción de `TRUE`

Fuente:

Datos numéricos

1. Enteros (Integers): enteros (32-bits)

2. “Doubles”: reales (64 bits) *[double precision vs. simple = 32 bits]*

Fuente:

Datos numéricos

1. Enteros (Integers): enteros (32-bits)

2. “Doubles”: reales (64 bits) *[double precision vs. simple = 32 bits]*

```
> x = sample(c(TRUE, FALSE), 10000, replace = TRUE)
> object.size(x) / 10000
4 bytes

> x = sample.int(999, 10000, replace = T)
> object.size(x) / 10000
4 bytes

> x = runif(10000)
> object.size(x) / 10000
8 bytes
```


Fuente:

Datos numéricos

Operaciones típicas:

- Aritmética: `+`, `-`, `*`, `/`.
- Aritmética modular: `%/%` (división entera) y `%%` (resto)

Fuente:

Datos numéricos

Operaciones típicas:

- Aritmética: `+`, `-`, `*`, `/`.
- Aritmética modular: `%/%` (división entera) y `%%` (resto)

Reciclaje: si hacemos cálculos en vectores de distinta longitud, R "recicla" (repite) el vector más corto para igualar al más largo.

```
> x = c(1,2,3)
> y = c(10, 100)
> x * y
[1]  10 200  30

Aviso:
In x * y :
  longitud de objeto mayor no es múltiplo de la longitud de uno menor
```


Fuente:

Datos numéricos

Los **enteros** se almacenan de manera exacta mientras el número esté dentro del rango permitido (¿cuántos int se pueden representar?)

Fuente: https://www.jasss.org/9/4/4.html

Datos numéricos

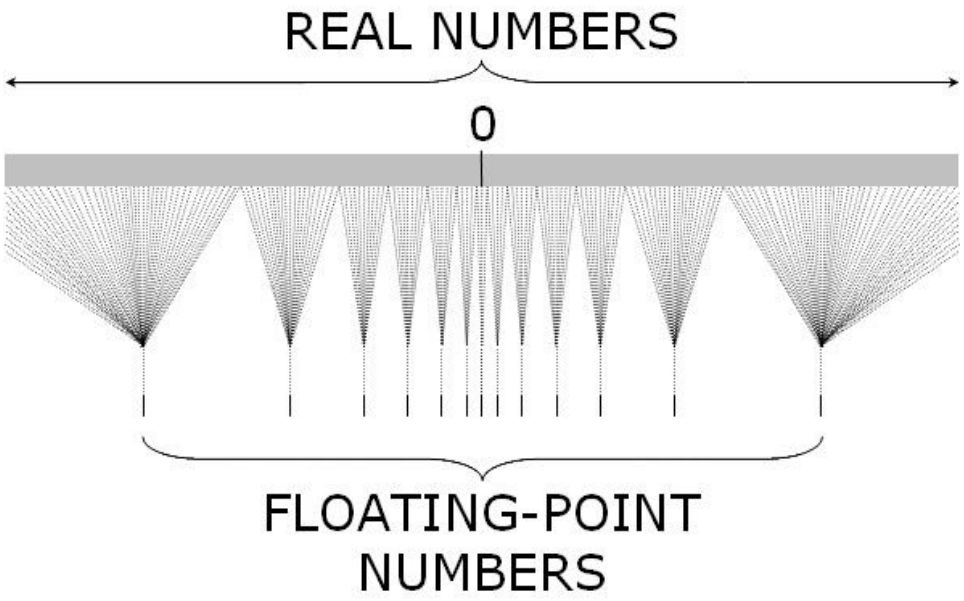
Los **enteros** se almacenan de manera exacta mientras el número esté dentro del rango permitido (e.g. 32 bits permite representar 2^32 enteros).

Fuente: https://www.jasss.org/9/4/4.html

Datos numéricos

Los **enteros** se almacenan de manera exacta mientras el número esté dentro del rango permitido (e.g. 32 bits permite representar 2^32 enteros).

Tener una cantidad limitada de bits para representar **números reales** (e.g. 64 bits) implica que solo podemos guardar una cantidad finita de números dentro de un rango (**precisión finita**)



Fuente: https://www.jasss.org/9/4/4.html

Datos numéricos

```
> x = sqrt(2) ^ 2
> x == 2
[1] FALSE
```


Fuente:

Datos numéricos

```
> x = sqrt(2) ^ 2
> x == 2
[1] FALSE
```

```
> print(x, digits = 22)
[1] 2.000000000000000444089
```


Fuente:

Datos numéricos

```
> x = sqrt(2) ^ 2
> x == 2
[1] FALSE
```

```
> print(x, digits = 22)
[1] 2.000000000000000444089
```

→ la representación de reales no es exacta

e.g. *(aproximación de sqrt(2))^2 != 2*

→ la igualdad matemática no siempre se cumple

→ Usar funciones de comparación con una tolerancia: `dplyr::near()`

Fuente:

Strings

Cadenas de caracteres: texto rodeados por comillas (`" "` o `' '`).

- Los caracteres se almacenan en la computadora según un **Encoding**: mapeo entre cadena de bits y caracter
- El estándar es **UTF-8**, que puede codificar casi cualquier caracter
- Cada caracter ocupa distinta cantidad de bytes en UTF-8

Fuente:

Strings

Cadenas de caracteres: texto rodeados por comillas (`" "` o `' '`).

- Los caracteres se almacenan en la computadora según un **Encoding**: mapeo entre cadena de bits y caracter
- El estándar es **UTF-8**, que puede codificar casi cualquier caracter
- Cada caracter ocupa distinta cantidad de bytes en UTF-8
- En general se usa el sistema hexadecimal (base 16) para representar los bits:

```
> charToRaw("a")
[1] 61
> charToRaw("á")
[1] c3 a1
> charToRaw("€")
[1] e2 82 ac
> charToRaw("🌍")
[1] f0 9f 8c 8e
```

- Dígitos hexa: 0-F
- 1 byte = 8 bits
  → 2^8 valores posibles
- 2 dígitos hexa. representan 1 byte:
  16^2 = 16 * 16 = 2^4 * 2^4 = 2^8

Fuente:

Strings

Operaciones:

- Longitud: `str_length()` cuenta los caracteres.
- Subconjunto: `str_sub()` extrae partes de un string por posición.
- Concatenación: `str_c()` o `str_glue()` unen cadenas de texto.
- Etc…: https://stringr.tidyverse.org/reference/index.html

Fuente:

Datos categóricos (Factores)

Variables categóricas con un conjunto fijo y conocido de valores posibles: niveles (*levels*)

- Permiten un ordenamiento basado en conceptos, no necesariamente alfabético e.g. meses
- Si un valor no está en los *levels*, se convierte a `NA` en lugar de aceptarse como una string válido

Fuente:

Datos categóricos (Factores)

Variables categóricas con un conjunto fijo y conocido de valores posibles: niveles (*levels*)

- Almacenan los datos como **enteros** que apuntan a un “diccionario” de *levels*→ más eficiente que usar strings

Fuente:

Datos categóricos (Factores)

Variables categóricas con un conjunto fijo y conocido de valores posibles: niveles (*levels*)

- Almacenan los datos como **enteros** que apuntan a un “diccionario” de *levels*→ más eficiente que usar strings

```
> x = sample(c("rené higuita", "romina", "ester píscole"), 10000, rep=T)
> object.size(x) / 10000
8 bytes
> x = as.factor(x)
> object.size(x) / 10000
4.1 bytes
```

- Cada string se guarda una vez en memoria, y el vector tiene punteros a posiciones en la memoria
- Cada puntero pesa 8 bytes
- Los factores se almacena como int (4 bytes) + una lookup table

Fuente:

Datos categóricos (Factores)

- `fct_reorder()` cambia el orden de los enteros basándose en otra variable (ej., ordenar "Color" por "Frecuencia" en lugar de alfabético) — útil para graficar
- `fct_recode()` cambia manualmente los nombres de las etiquetas
- `fct_lump()` *colapsa* los niveles menos comunes en un solo grupo "Otros".
- https://forcats.tidyverse.org/

Fuente:

Datetime

1. Date: un día específico (resolución: días)

2. Time: una hora del día (resolución: segundos)

3. Date-time: un instante específico (resolución: al segundo más cercano).
Se imprime como `<dttm>` (o POSIXct en R base)

Los datos a veces llegan como *strings*
Los podemos *parsear* en estos tipos usando estándares
(e.g. `YYYY-MM-DD`) o *helpers* como `ymd()`

Fuente:

Datetime

Las computadoras almacenan el tiempo como un número que representa un desplazamiento desde “Unix Epoch”: 1 de enero de 1970.

- Fechas: número de días desde Unix Epoch
  e.g. `as_date(365*10 + 2)` -> "1980-01-01".
- Date-times: número de segundos desde Unix Epoch
  e.g. `as_datetime(36000)` -> "1970-01-01 10:00:00 UTC".

→ La **aritmética de fechas** es aritmética de números

Fuente:

Datetime

Sumar "un día" o "un año" es difícil debido a los años bisiestos y al Horario de Verano (Daylight Saving Time o DST).

Hay funciones que nos ayudan a controlar esto:

1. Durations: Número exacto de segundos.
   e.g. sumar 86,400 segundos (un día) puede cambiar la hora del reloj si ocurrió DST
2. Periods: Unidades humanas (días, meses).
   e.g. sumar `1 day` mantiene la hora del reloj, incluso a través de los límites del DST.
3. Intervals: Una duración con un punto de inicio y fin específicos.
   e.g. para determinar superposiciones o divisiones precisas (ej., ¿cuántos días hay en *este* año en específico?).

Fuente:

Valores faltantes

Representan lo desconocido: `NA` (Not Available).

No es cero; es la ausencia de información.

- `5 + NA` →
- `NA > 5` →
- `NA == NA` →

Fuente:

Valores faltantes

Representan lo desconocido: `NA` (Not Available).

No es cero; es la ausencia de información.

Son contagiosos:

- `5 + NA` → `NA`.
- `NA > 5` → `NA`.
- `NA == NA` → `NA` (no sabemos si un valor desconocido es igual a otro desconocido)
  → `is.na(x)` devuelve `TRUE` o `FALSE` indicando si un valor falta.
- `NA` pueden romper las estadísticas de resumen

Fuente:

Valores faltantes

Faltante Explícito (`NA`):

- La celda existe en la tabla, pero no contiene ningún valor.
- ej. Un dataset que lista los precios de las acciones para cada trimestre, donde el Q4 de 2020 contiene `NA` en el precio

Fuente:

Valores faltantes

Faltante Explícito (`NA`):

- La celda existe en la tabla, pero no contiene ningún valor.
- ej. Un dataset que lista los precios de las acciones para cada trimestre, donde el Q4 de 2020 contiene `NA` en el precio

Faltante Implícito:

- El registro en sí está completamente ausente en los datos.
- ej. Un dataset que lista los precios de las acciones donde la fila para el Q1 de 2021 falta

Fuente:

Valores faltantes

`complete()` expone los valores faltantes implícitos al generar todas las combinaciones posibles de variables (ej., asegurando que exista cada par de año-trimestre) y llenando las nuevas filas con `NA`.

`anti_join()` sirve para encontrar datos faltantes implícitos al comparar una tabla con una lista "maestra" (ej., encontrar qué aviones en un registro de vuelos no existen en el registro de aviones).

Fuente:

Valores faltantes

`NaN` (Not a Number)

Fuente:

Valores faltantes

`NaN` (Not a Number)

Tipo distinto que resulta de matemática imposible (ej., 0/0), pero generalmente se comporta como `NA`.

Fuente:

Valores faltantes

Comprender *por qué* faltan datos dicta cómo debes manejarlos.

1. Estructural: El valor falta porque lógicamente no puede existir.

* *Ejemplo:* Un "Año de Ratificación de Tratado" falta para un país que nunca firmó el tratado.

2. MCAR (Faltante Completamente al Azar): No existe ningún patrón; la falta de datos no está relacionada con los datos (ej., se perdió una página aleatoria de una encuesta).

3. MAR (Faltante al Azar): La falta de datos está relacionada con las variables observadas (ej., la falta de datos de ingresos se correlaciona con la edad, la cual *sí* tienes).

4. MNAR (Faltante No al Azar): Existe un patrón en el que la falta de datos está relacionada con el valor en sí.

* *Ejemplo:* Países pobres sin datos de PIB porque carecen de los recursos para medirlo.

Fuente:

Valores faltantes

A veces, `NA` representa un valor conocido, a veces cero

- ej. Si una columna de "Comisión" tiene un `NA`, podría significar $0.
- `coalesce()` reemplaza los `NA`s con un valor específico.

Última Observación Llevada Hacia Adelante (LOCF):

- Ej. en el ingreso de datos o en series de tiempo donde un valor implica "igual que arriba".
- `fill()` reemplaza los valores faltantes con el valor no faltante más reciente de una fila anterior.

A veces, los valores faltantes se ocultan como números específicos (ej., "99" o "-999")

- Pueden convertirse a `NA` usando `na_if()`

Fuente:

Valores faltantes

Antes de limpiar, explorar la distribución de faltantes

→ No eliminar ciegamente!
Puede introducir sesgos si los datos no faltan completamente al azar

→ Investigar *por qué* faltan los datos

Imputación: predecir los valores faltantes basándose en la información presente en otras variables.

Fuente:

4. Convenciones de Nomenclatura Consistentes

* Usar `snake_case` (minúsculas con guiones bajos, ej., `term_length`) para variables y funciones.

* evitar usar puntos (ej., `term.length`) en los nombres — R usa puntos para su sistema orientado a objetos (métodos S3), lo cual puede llevar a conflictos.

Fuente:

Lecturas de esta clase

Magallanes Reyes 2022 (cap. 2)
Weidmann 2023 (cap. 3)
Wickham et al. 2023 (ch. 3-5 & 18-19)

Complementaria

Baumer et al. 2024 (cap. 4-6)
Urdinez & Cruz 2020 (cap. 2 & 11)
Weidmann 2023 (cap. 6 & 7)
Wickham et al. 2023 (cap. 12-17)

Extras

Joins

Los **filtering joins** afectan a qué observaciones se mantienen, pero no agregan nuevas columnas.

Semi join mantiene todas las filas de la primera tabla que tienen una coincidencia en la segunda tabla.
e.g. filtrar una tabla de aeropuertos para mostrar solo los que aparecen como destino en la tabla de vuelos.

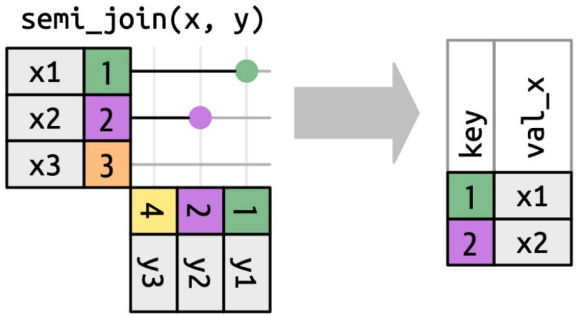


Figure 19.10: In a semi-join it only matters that there is a match; otherwise values in `y` don't affect the output.

Fuente: https://r4ds.hadley.nz

Joins

Los **filtering joins** afectan a qué observaciones se mantienen, pero no agregan nuevas columnas.

Semi join mantiene todas las filas de la primera tabla que tienen una coincidencia en la segunda tabla.
e.g. filtrar una tabla de aeropuertos para mostrar solo los que aparecen como destino en la tabla de vuelos.

Anti-join mantiene todas las filas en la primera tabla que *no* tienen coincidencia en la segunda tabla.
e.g. encontrar qué vuelos tienen un `tailnum` que falta en el registro de `planes`.

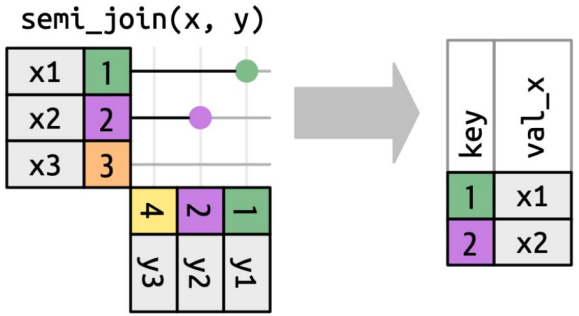


Figure 19.10: In a semi-join it only matters that there is a match; otherwise values in `y` don't affect the output.

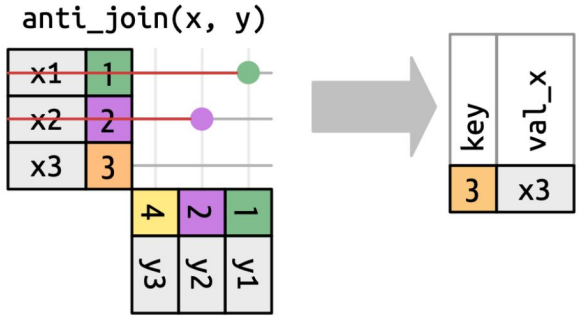
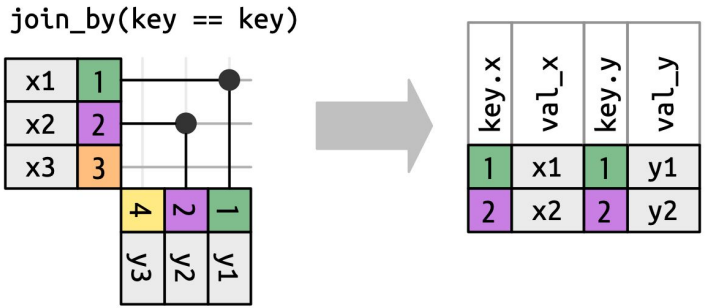


Figure 19.11: An anti-join is the inverse of a semi-join, dropping rows from `x` that have a match in `y`.

Fuente: https://r4ds.hadley.nz

Non-equi joins

En lugar de emparejar por igualdad (`x == y`), hacerlo por proximidad.



Fuente:

Non-equi joins

Los **Inequality Joins** usan condiciones como `<` o `>=`.

E.g. emparejar presidentes con datos económicos anuales *entre* los años de inicio y fin del presidente (`year >= startyear` & `year <= endyear`)

este se llama también un overlap join

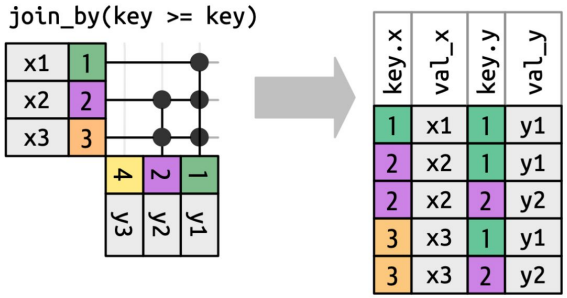


Figure 19.13: A non-equi join where the `x` key must be greater than or equal to the `y` key. Many rows generate multiple matches.

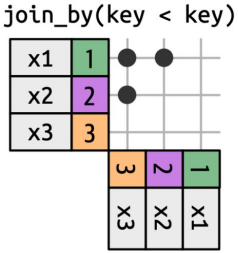


Figure 19.15: An inequality join where `x` is joined to `y` on rows where the key of `x` is less than the key of `y`. This makes a triangular shape in the top-left corner.

Fuente: https://r4ds.hadley.nz

Non-equi joins

Los **Rolling Joins** encuentran la coincidencia *más cercana*.

e.g. tabla de presidentes y una tabla de eventos políticos. Queremos emparejar cada presidente con los eventos *más cercanos* que ocurrieron antes o durante su mandato

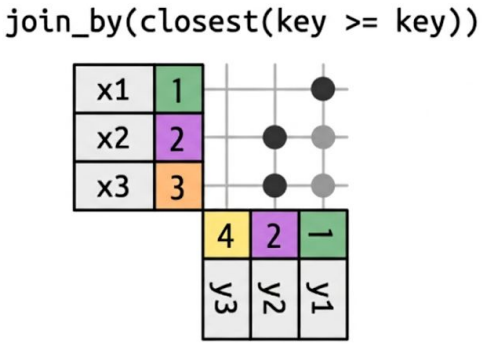


Figure 19.16: A rolling join is similar to a greater-than-or-equal inequality join but only matches the first value.

Fuente: https://r4ds.hadley.nz, corregido con Nano Banana

Non-equi joins

El **Cross Join** genera todas las combinaciones posibles de filas (producto cartesiano).

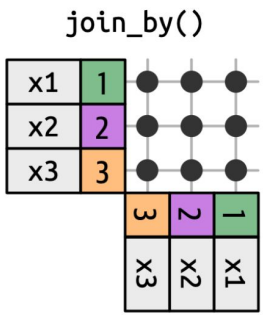


Figure 19.14: A cross join matches each row in `x` with every row in `y`.

```r
df <- tibble(name = c("John", "Simon", "Tracy", "Max"))
df |> cross_join(df)
#> # A tibble: 16 × 2
#>   name.x name.y
#>   <chr>  <chr>
#> 1 John   John
#> 2 John   Simon
#> 3 John   Tracy
#> 4 John   Max
#> 5 Simon  John
#> 6 Simon  Simon
#> # ℹ 10 more rows
```


Fuente: https://r4ds.hadley.nz